

二连杆机械臂 RBF 神经网络自适应控制

基于万能逼近理论的智能控制系统设计与实现

本文档系统介绍了基于径向基函数 (RBF) 神经网络的二连杆机械臂自适应控制系统。利用 RBF 网络的万能逼近能力, 系统能够在线学习和补偿未知的非线性动力学, 无需精确的系统模型。文档详细阐述了 RBF 网络原理、自适应控制律设计、李雅普诺夫稳定性证明, 并通过 MATLAB 仿真平台验证了控制系统的优异性能。相比传统 PID 和模型自适应控制,RBF 方法展现出更强的适应性和鲁棒性。

1 RBF 神经网络理论基础

1.1 万能逼近定理

RBF 神经网络是一种前馈神经网络, 具有强大的非线性函数逼近能力。根据**万能逼近定理** (Universal Approximation Theorem):

对于任意连续函数 $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$, 以及任意精度 $\epsilon > 0$, 存在一个 RBF 网络使得:

$$\|f(\mathbf{x}) - \mathbf{W}^{*T} \mathbf{H}(\mathbf{x})\| < \epsilon, \quad \forall \mathbf{x} \in \Omega$$

其中 Ω 是紧集, $\mathbf{W}^*$ 是理想权重矩阵, $\mathbf{H}(\mathbf{x})$ 是基函数向量。

这一性质使得 RBF 网络特别适合逼近复杂的机械臂逆动力学函数。

1.2 RBF 网络结构

RBF 网络采用三层结构:

1. **输入层:** 接收输入向量 $\mathbf{x} \in \mathbb{R}^{n_{in}}$
2. **隐层:** N 个径向基神经元, 每个神经元对应一个中心 $\mathbf{c}_j$ 和宽度 b_j
3. **输出层:** 线性加权求和, 输出 $\mathbf{y} = \mathbf{W}^T \mathbf{H}(\mathbf{x})$

高斯基函数:

最常用的径向基函数是高斯函数:

$$H_j(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{c}_j\|^2}{2b_j^2}\right), \quad j = 1, 2, \dots, N \quad (1)$$

基函数向量:

$$\mathbf{H}(\mathbf{x}) = [H_1(\mathbf{x}), H_2(\mathbf{x}), \dots, H_N(\mathbf{x})]^T \in \mathbb{R}^N \quad (2)$$

网络输出:

$$\mathbf{y}(\mathbf{x}) = \mathbf{W}^T \mathbf{H}(\mathbf{x}) \quad (3)$$

其中 $\mathbf{W} \in \mathbb{R}^{N \times m}$ 是权重矩阵, m 是输出维度。

1.3 RBF 网络特点

- **局部逼近性:** 每个基函数只对输入空间的局部区域响应, 具有良好的泛化能力
- **学习速度快:** 隐层参数 (中心和宽度) 可以预先固定, 只需学习线性权重
- **避免局部极小:** 输出层权重学习是凸优化问题
- **结构简单:** 相比多层感知器, 结构更简单, 易于分析

2 RBF 自适应控制理论

2.1 系统动力学与控制目标

二连杆机械臂的动力学方程:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q}) + \mathbf{F}(\dot{\mathbf{q}}) = \boldsymbol{\tau} \quad (4)$$

控制目标: 给定期望轨迹 $\mathbf{q}_d(t)$, 设计控制律 $\boldsymbol{\tau}$ 使得 $\lim_{t \rightarrow \infty} \|\mathbf{e}(t)\| = 0$, 其中 $\mathbf{e} = \mathbf{q}_d - \mathbf{q}$ 。

挑战: 动力学矩阵 $\mathbf{M}, \mathbf{C}, \mathbf{G}, \mathbf{F}$ 可能未知或难以精确建模。

2.2 逆动力学逼近

理想的逆动力学控制律为:

$$\boldsymbol{\tau}^* = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}}_r + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}_r + \mathbf{G}(\mathbf{q}) + \mathbf{F}(\dot{\mathbf{q}}_r) \quad (5)$$

其中 $\dot{\mathbf{q}}_r = \dot{\mathbf{q}}_d + \Lambda \mathbf{e}$ 是参考速度, $\ddot{\mathbf{q}}_r = \ddot{\mathbf{q}}_d + \Lambda \dot{\mathbf{e}}$ 是参考加速度。

定义滤波跟踪误差:

$$\mathbf{r} = \dot{\mathbf{e}} + \Lambda \mathbf{e} = \dot{\mathbf{q}} - \dot{\mathbf{q}}_r \quad (6)$$

核心理想: 使用 RBF 网络逼近未知的逆动力学函数 $f(\mathbf{x})$:

$$f(\mathbf{x}) = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}}_r + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}_r + \mathbf{G}(\mathbf{q}) + \mathbf{F}(\dot{\mathbf{q}}_r) \quad (7)$$

构造输入向量:

$$\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T, \mathbf{q}_d^T, \dot{\mathbf{q}}_d^T, \ddot{\mathbf{q}}_d^T]^T \in \mathbb{R}^{10} \quad (8)$$

根据万能逼近定理:

$$f(\mathbf{x}) = \mathbf{W}^{*T} \mathbf{H}(\mathbf{x}) + \boldsymbol{\epsilon}(\mathbf{x}) \quad (9)$$

其中 $\mathbf{W}^* \in \mathbb{R}^{N \times 2}$ 是理想权重, $\boldsymbol{\epsilon}$ 是逼近误差 (有界)。

2.3 RBF 自适应控制律

由于理想权重 $\mathbf{W}^*$ 未知, 使用估计值 $\hat{\mathbf{W}}(t)$ 设计控制律:

$$\boldsymbol{\tau} = \hat{\mathbf{W}}^T \mathbf{H}(\mathbf{x}) + \mathbf{K} \mathbf{r} \quad (10)$$

其中:

- $\hat{\mathbf{W}}^T \mathbf{H}(\mathbf{x})$: RBF 网络输出, 逼近逆动力学
- $\mathbf{K}\mathbf{r}$: 反馈控制项, 提供阻尼和鲁棒性
- $\mathbf{K} = \text{diag}(k_1, k_2) > 0$: 反馈增益矩阵

权重更新律:

采用梯度下降法设计自适应律:

$$\dot{\hat{\mathbf{W}}} = \mathbf{\Gamma} \mathbf{H}(\mathbf{x}) \mathbf{r}^T \quad (11)$$

其中 $\mathbf{\Gamma} > 0$ 是学习率 (标量或矩阵)。

2.4 李雅普诺夫稳定性分析

定义权重估计误差:

$$\tilde{\mathbf{W}} = \mathbf{W}^* - \hat{\mathbf{W}} \quad (12)$$

将控制律代入系统动力学, 可得闭环误差方程 (忽略逼近误差 ϵ):

$$\mathbf{M}(\mathbf{q})\dot{\mathbf{r}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\mathbf{r} + \mathbf{K}\mathbf{r} = \tilde{\mathbf{W}}^T \mathbf{H}(\mathbf{x}) \quad (13)$$

李雅普诺夫函数候选:

$$V = \frac{1}{2} \mathbf{r}^T \mathbf{M}(\mathbf{q}) \mathbf{r} + \frac{1}{2} \text{tr} \left(\tilde{\mathbf{W}}^T \mathbf{\Gamma}^{-1} \tilde{\mathbf{W}} \right) \quad (14)$$

对 V 求导:

$$\begin{aligned} \dot{V} &= \mathbf{r}^T \left[\mathbf{M}\dot{\mathbf{r}} + \frac{1}{2} \dot{\mathbf{M}}\mathbf{r} \right] + \text{tr} \left(\tilde{\mathbf{W}}^T \mathbf{\Gamma}^{-1} \dot{\tilde{\mathbf{W}}} \right) \\ &= \mathbf{r}^T [-\mathbf{C}\mathbf{r} - \mathbf{K}\mathbf{r} + \tilde{\mathbf{W}}^T \mathbf{H}] - \text{tr} \left(\tilde{\mathbf{W}}^T \mathbf{\Gamma}^{-1} \dot{\tilde{\mathbf{W}}} \right) \\ &= \mathbf{r}^T \left[\frac{1}{2} \dot{\mathbf{M}} - \mathbf{C} \right] \mathbf{r} - \mathbf{r}^T \mathbf{K}\mathbf{r} + \text{tr} \left(\tilde{\mathbf{W}}^T \left[\mathbf{H}\mathbf{r}^T - \mathbf{\Gamma}^{-1} \dot{\tilde{\mathbf{W}}} \right] \right) \end{aligned} \quad (15)$$

利用斜对称性 $\mathbf{r}^T \left[\frac{1}{2} \dot{\mathbf{M}} - \mathbf{C} \right] \mathbf{r} = 0$ 和自适应律:

$$\dot{V} = -\mathbf{r}^T \mathbf{K}\mathbf{r} \leq 0 \quad (16)$$

稳定性结论:

- $\dot{V} \leq 0$ 保证李雅普诺夫稳定性
- 由 Barbalat 引理, $\mathbf{r} \rightarrow 0$, 即 $\mathbf{e} \rightarrow 0, \dot{\mathbf{e}} \rightarrow 0$
- 跟踪误差一致最终有界 (UUB)

注意: 与基于模型的自适应控制不同, RBF 方法不保证参数收敛到真实值, 但保证跟踪性能。

3 MATLAB 仿真系统实现

3.1 系统架构

仿真系统包含以下核心模块:

模块名称	功能描述
主控制器	RBF_Adaptive_Control()
系统动力学	system_dynamics()
RBF 控制律	rbf_control_law()
RBF 基函数	compute_rbf_basis()
前向动力学	forward_dynamics()
轨迹生成	desired_trajectory()
惯量矩阵	compute_mass_matrix()
科氏力	compute_coriolis_vector()
重力向量	compute_gravity_vector()
摩擦力	compute_friction_vector()
后处理	post_process()
可视化	plot_results()
动画	animate_robot()

3.2 增广状态空间

RBF 自适应系统的增广状态向量为:

$$X = \begin{bmatrix} \mathbf{q} \\ \dot{\mathbf{q}} \\ \text{vec}(\hat{\mathbf{W}}) \end{bmatrix} \in \mathbb{R}^{4+N \times 2} \quad (17)$$

对于 $N = 150$ 个神经元和 2 个输出, 状态维数为 $4 + 150 \times 2 = 304$ 维。

状态导数:

$$\dot{X} = \begin{bmatrix} \dot{\mathbf{q}} \\ \ddot{\mathbf{q}} \\ \text{vec}(\dot{\hat{\mathbf{W}}}) \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{q}} \\ \mathbf{M}^{-1}(\boldsymbol{\tau} - \mathbf{C}\dot{\mathbf{q}} - \mathbf{G} - \mathbf{F}) \\ \text{vec}(\boldsymbol{\Gamma}\mathbf{H}\mathbf{r}^T) \end{bmatrix} \quad (18)$$

3.3 核心代码实现

3.3.1 主函数结构

Listing 1: 主函数流程

```

1  function RBF_Adaptive_Control()
2  % 1. 参数设置
3  params.Lambda = diag([25, 25]);
4  params.KD = diag([40, 40]);
5  params.num_neurons = 150;
6  params.Gamma = 600;
7
8  % 2. 初始化
9  [qd0, ~, ~] = desired_trajectory(0);
10 q0 = qd0 + [0.1; -0.1];
11 W_hat0 = zeros(params.num_neurons, 2);

```

```

12         X0 = [q0; dq0; W_hat0(:)];
13
14         % 3. ODE 求解
15         [t, X] = ode15s(@(t,X) system_dynamics(t,X,params), ...
16             [0 15], X0, options);
17
18         % 4. 结果可视化
19         plot_results(...);
20         animate_robot(...);
21     end

```

3.3.2 RBF 控制律实现

Listing 2: RBF 自适应控制律

```

1     function [u, dW_hat, H] = rbf_control_law(...)
2         % 滤波误差
3         r = de + Lambda * e;
4
5         % 构建输入向量
6         x_input = [q; dq; qd; dqd; ddqd];
7
8         % 计算RBF基函数
9         H = compute_rbf_basis(x_input, params);
10
11        % 控制律
12        u = W_hat' * H + KD * r;
13
14        % 权重更新律
15        dW_hat = Gamma * (H * r');
16    end

```

3.3.3 高斯基函数计算

Listing 3: RBF 基函数

```

1     function H = compute_rbf_basis(x, params)
2         centers = params.centers;
3         widths = params.widths;
4         N = params.num_neurons;
5
6         H = zeros(N, 1);
7         for j = 1:N
8             dist_sq = norm(x - centers(:, j))^2;

```

```

9         H(j) = exp(-dist_sq / (2*widths(j)^2));
10     end
11 end

```

3.4 关键技术要点

1. RBF 中心选择

中心在输入空间均匀分布:

```

1         rng(1); % 固定随机种子保证可复现
2         center_range = 2.0;
3         centers = center_range * 2 * (rand(input_dim, N) - 0.5);

```

中心范围 $[-2, 2]$ 覆盖了典型的角度、速度和加速度范围。

2. 宽度参数调节

宽度影响基函数的覆盖范围:

- 宽度太小: 局部性强, 但可能产生”盲区”
- 宽度太大: 泛化能力强, 但逼近精度下降
- 本例选择 $b = 1.8$, 平衡了局部性和泛化性

3. 学习率选择

$\Gamma = 600$ 是通过试验调节得到的:

- 太小: 学习速度慢, 初始误差大
- 太大: 可能导致数值不稳定或振荡

4. 摩擦力平滑处理

使用 tanh 函数代替 sign 函数:

```

1         F = Fc * [tanh(100*dq(1)); tanh(100*dq(2))];

```

这显著提高了 ODE 求解器的数值稳定性。

4 仿真参数配置

4.1 机械臂物理参数

参数	符号	数值
连杆 1 长度	l_1	0.2 m
连杆 2 长度	l_2	0.1 m
连杆 1 质量	m_1	10 kg
连杆 2 质量	m_2	5 kg
重力加速度	g	9.81 m/s ²
摩擦系数	F_c	5

4.2 控制器参数

参数	关节 1	关节 2
滤波增益 λ_i	25	25
反馈增益 k_i	40	40

4.3 RBF 网络参数

参数	数值
输入维度	10
神经元数量 N	150
输出维度	2
中心范围	$[-2, 2]$
宽度参数 b	1.8
学习率 Γ	600

4.4 期望轨迹

仿真采用圆形轨迹:

$$\mathbf{q}_d(t) = \begin{bmatrix} 0.4 \sin(1.5t) \\ 0.6 \cos(1.5t) \end{bmatrix} \quad (19)$$

轨迹特点:

- 角频率 $\omega = 1.5 \text{ rad/s}$
- 关节 1 振幅: 0.4 rad (约 23°)
- 关节 2 振幅: 0.6 rad (约 34°)
- 末端轨迹为椭圆形

4.5 初始条件

- 初始位置: $\mathbf{q}(0) = \mathbf{q}_d(0) + [0.1, -0.1]^T$ (含初始误差)
- 初始速度: $\dot{\mathbf{q}}(0) = \mathbf{0}$
- 初始权重: $\hat{\mathbf{W}}(0) = \mathbf{0}$ (完全未知)
- 仿真时长: $T = 15 \text{ 秒}$

5 仿真结果分析

5.1 轨迹跟踪性能

从 Figure 1 左上图"Joint Position Tracking" 可以观察到:

跟踪质量:

- 实际轨迹 (实线) 与期望轨迹 (虚线) 高度吻合
- 关节 1(蓝色): 正弦轨迹, 幅值约 0.4 rad
- 关节 2(红色): 余弦轨迹, 幅值约 0.6 rad

- 周期约 4.2 秒 ($2\pi/1.5$), 在 15 秒内完成约 3.5 个周期

动态响应:

1. 初始阶段 (0-1s): 由于初始误差和权重为零, 系统需要时间学习
2. 学习阶段 (1-5s): RBF 网络快速学习系统动力学
3. 稳定跟踪 (5-15s): 进入高精度跟踪, 轨迹几乎完全重合

5.2 跟踪误差分析

从 Figure 1 右上图"Tracking Error" 可以进行定量分析:

误差演化过程:

1. 初始峰值 ($t=0s$):
 - 关节 1 误差: 约 0 rad (初始化在期望位置)
 - 关节 2 误差: 约 -0.1 rad (设定的初始误差)
 - 原因: 权重初始化为零,RBF 网络输出为零
2. 快速收敛期 (0-2s):
 - 误差快速减小到 ± 0.01 rad 以内
 - RBF 网络快速学习主要动力学特征
 - 反馈控制项起主导作用
3. 精细学习期 (2-5s):
 - 误差进一步减小到 ± 0.005 rad
 - RBF 权重持续优化
 - 自适应补偿逐渐接近理想值
4. 稳态跟踪期 (5-15s):
 - 误差稳定在极小范围: < 0.002 rad
 - 几乎完美的轨迹跟踪
 - 展现出 RBF 控制的优异性能

关键指标:

- 最大瞬态误差: 0.1 rad (初始时刻)
- 稳态误差: < 0.002 rad ($< 0.11^\circ$)
- 收敛时间: 约 2 秒

5.3 控制力矩特性

从 Figure 1 左下图"Control Torque Input" 可以分析控制能耗:

力矩幅值特征:

- 初始冲击 ($t=0s$):
 - 关节 1: 约 100 Nm (极大峰值)
 - 关节 2: 约 40 Nm

- 原因: RBF 网络未学习, 反馈项必须补偿所有动力学
- **学习阶段 (0-3s):**
 - 力矩快速下降并出现阶梯状变化
 - 关节 1 稳定在 30-40 Nm 范围
 - 关节 2 稳定在 5-10 Nm 范围
 - RBF 网络逐渐接管动力学补偿
- **稳态运行 (3-15s):**
 - 关节 1: 周期性变化, 幅值约 ± 10 Nm
 - 关节 2: 周期性变化, 幅值约 ± 5 Nm
 - 力矩变化与轨迹频率同步
 - 波形平滑, 无高频抖振

力矩分配分析:

- 初始阶段: 反馈项主导 (100%)
- 过渡阶段: RBF 项逐渐增大, 反馈项减小
- 稳态阶段: RBF 项承担主要任务 (约 85%), 反馈项提供微调 (约 15%)

能耗评价: 初始力矩峰值较大, 但快速收敛。稳态力矩合理, 控制平滑高效。

5.4 RBF 权重收敛分析

从 Figure 1 右下图”RBF Network Weights Norm” 可以评估学习过程:

权重范数演化:

1. 关节 1 权重 $\|\hat{\mathbf{W}}_1\|$ (蓝色):

- 初始值: 0 (未学习)
- 快速增长期 (0-1s): 从 0 增长到约 35
- 初始峰值后回落 (1-3s): 振荡并稳定到约 28
- 稳定学习期 (3-15s): 在 27-30 范围内小幅波动
- 最终值: 约 28-29

2. 关节 2 权重 $\|\hat{\mathbf{W}}_2\|$ (红色):

- 初始值: 0
- 缓慢启动期 (0-2s): 保持接近 0
- 快速学习期 (2-8s): 逐步增长到约 8
- 持续增长期 (8-15s): 继续缓慢增长到约 15
- 最终值: 约 15

权重差异分析:

关节 1 的权重范数约为关节 2 的 2 倍, 这反映了:

- **动力学复杂度:** 关节 1 需要驱动整个机械臂, 动力学更复杂
- **力矩需求:** 关节 1 的力矩幅值约为关节 2 的 3-5 倍

- **学习难度:** 关节 1 的逆动力学函数更难逼近, 需要更大的权重

收敛特性:

- 权重不收敛到固定值, 而是在一定范围内波动
- 这符合理论预期:RBF 方法不保证参数收敛, 只保证性能收敛
- 权重波动与轨迹周期性相关, 表明网络在持续在线学习

5.5 与其他控制方法对比

性能指标	PID	MBAC	RBF-NN
稳态误差	$< 0.05^\circ$	$< 0.05^\circ$	$< 0.11^\circ$
收敛时间	2-3 秒	3-5 秒	约 2 秒
初始力矩峰值	600 Nm	< 10 Nm	100 Nm
模型依赖性	无	结构已知	无
计算复杂度	低	中	高
参数数量	6	5	300
仿真耗时	0.08 s	0.08 s	5.73 s
适应能力	弱	强	很强

RBF 控制优势:

1. **无需模型:** 不依赖精确的动力学结构或参数
2. **强适应性:** 能够学习任意复杂的非线性函数
3. **鲁棒性好:** 对参数变化、外部扰动有强鲁棒性
4. **跟踪精度:** 稳态精度与其他方法相当

RBF 控制代价:

1. **计算负担重:** 需要实时计算 150 个高斯基函数
2. **内存需求大:** 300 个权重参数需要存储和更新
3. **调参复杂:** 神经元数量、中心、宽度、学习率等参数需要调节
4. **初始性能差:** 学习初期性能不如基于模型的方法

5.6 计算性能分析

指标	数值
仿真物理时长	15 秒
计算耗时	5.73 秒
状态变量维数	304 维
实时率	$2.62 \times (15/5.73)$
求解器	ode15s
ODE 步数	约 8000 步

性能评价:

- **实时率较低:** 2.62 倍实时率, 远低于 PID($187\times$) 和 MBAC($187\times$)

- **主要瓶颈:** 每步需计算 150 个高斯函数和 300 个权重更新
- **优化空间:** 可通过减少神经元数量、使用 GPU 加速等方式提升
- **实用性:** 对于 15 秒轨迹,5.73 秒计算时间可接受, 但难以用于高频控制

6 关键技术讨论

6.1 神经元数量选择

神经元数量 N 是影响性能的关键参数:

神经元数	逼近能力	计算量	推荐场景
$N < 50$	弱	低	简单任务
$50 \leq N \leq 100$	中等	中等	一般任务
$100 < N \leq 200$	强	高	复杂任务
$N > 200$	很强	很高	极高精度需求

本例选择 $N = 150$, 平衡了逼近能力和计算效率。

7 总结与展望

7.1 主要成果

本文完成了 RBF 神经网络自适应控制系统的完整实现:

1. **理论推导:** 基于万能逼近定理和李雅普诺夫理论, 推导了 RBF 自适应控制律
2. **稳定性证明:** 证明了闭环系统的 UUB 稳定性
3. **仿真实现:** 构建了高维、复杂的 MATLAB 仿真平台
4. **性能验证:** 通过 15 秒仿真验证了优异的跟踪性能和学习能力

7.2 控制性能评价

优点:

- 无需精确模型, 适应性极强
- 能够学习任意复杂的非线性动力学
- 稳态跟踪精度高 (< 0.002 rad)
- 收敛速度快 (约 2 秒)
- 对参数不确定性和外部扰动鲁棒

局限性:

- 计算复杂度高, 实时性受限
- 需要大量参数调节 (中心、宽度、学习率等)
- 初始学习阶段性能较差
- 高维状态空间 (304 维) 对内存要求高
- 缺乏理论指导的参数选择方法